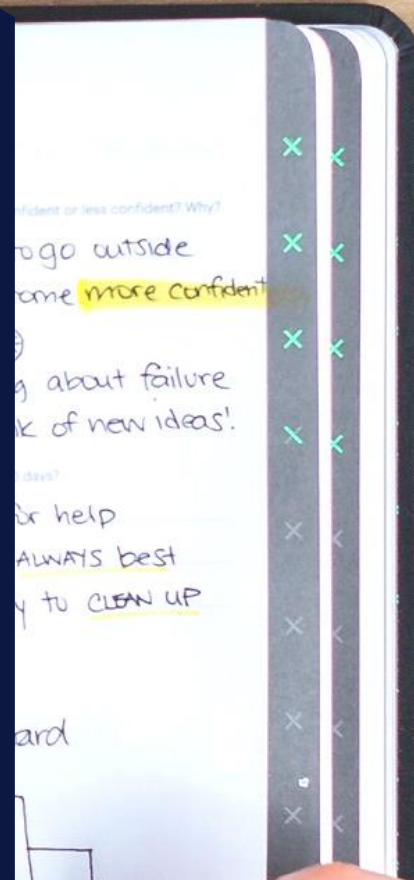




Tarea # 1

Unidad 2. Programación
del lado del servidor



Actividades

Actividad: Desarrollo de una aplicación web de gestión de productos con autenticación y chat en tiempo real

Objetivo General

Desarrollar una aplicación web funcional y segura, utilizando Node.js, Express, MongoDB, y Socket.io, que permita aplicar de forma integrada los conceptos de programación del lado del servidor, operaciones HTTP, gestión de datos, manejo de archivos y sesiones, así como comunicación en tiempo real, fortaleciendo las competencias técnicas del estudiante en el diseño y construcción de soluciones web modernas.

Requisitos Previos

- Mantener la organización de los archivos una estructura clara
- El código resultante debe ser:
 - Entregado comprimido en un archivo .zip
 - Subido a un repositorio público en GitHub con instrucciones mínimas de uso (README.md)

Descripción general

El estudiante desarrollará una aplicación web completa llamada "MiInventarioExpress", que permita la gestión de productos (alta, baja, modificación, consulta), el almacenamiento de dichos datos en MongoDB, autenticación básica de usuarios, carga de imágenes de productos y un módulo de chat en tiempo real para interacción entre usuarios administradores.

Se evaluará el uso adecuado del modelo MVC, la implementación de rutas con Express, persistencia con MongoDB (vía Mongoose), validación de formularios, carga de archivos con Multer, sesiones de usuario y comunicación en tiempo real con Socket.io.

Tareas Obligatorias

No.	Tarea
-----	-------

1	Crear un repositorio en GitHub Realizar al menos tres commits en fechas diferentes . Generar un archivo README.md indicando los datos del estudiante, las funcionalidades implementadas e instrucciones básicas de uso. Coloca la URL del repositorio de GitHub en un archivo readme.txt dentro del ZIP entregado .
2	Inicializar el proyecto: configuración básica con npm init, instalación de dependencias. Definir el archivo .gitignore
3	Crear estructura de carpetas siguiendo el patrón MVC (models, routes, views, public, uploads)
4	Implementar CRUD completo para productos (crear, listar, actualizar, eliminar) usando Express y MongoDB
5	Diseñar e implementar el esquema de Producto en Mongoose con campos: nombre, precio, descripción, imagen
6	Habilitar carga de imágenes para cada producto usando Multer (con validación de tipo y tamaño)
7	Implementar autenticación de usuarios (login/logout), sesiones con express-session y contraseñas con bcrypt
8	Agregar validaciones a formularios con express-validator y control de errores para entradas inválidas
9	Diseñar vistas con Handlebars para listar productos, formulario de creación/edición y página de login
10	Integrar un módulo de chat en tiempo real entre usuarios autenticados usando Socket.io
11	Pruebas manuales y validación de funcionalidades usando Postman, navegador y consola

INSTRUCCIONES PARA LA REALIZACIÓN DE LA ACTIVIDAD DE LA UNIDAD 1

1. Crear un repositorio en GitHub y trabajar de forma síncrona con el proyecto
2. Generar un archivo README.md indicando los datos del estudiante, las funcionalidades implementadas e instrucciones básicas de uso

- Coloca la URL de su repositorio de GitHub en un archivo readme.txt dentro de la carpeta principal del proyecto
- Una vez concluido su desarrollo, **comprimir la carpeta en un archivo ZIP y enviar al AVAC** con el siguiente formato “Actividad2-Apellido1-Nombre1.zip”.

Rúbrica:

Criterios	Bajo (0%)	Regular (30%)	Medio (60%)	Alto (100%)	Sub-puntaje máximo
1. Funcionalidad Ejecuta correctamente las tareas definidas	No cumple tareas o presentan fallos graves. 0	Cumple hasta 4 tareas de forma parcial o incompleta. 1.8	Cumple entre 5 y 9 tareas con funcionamiento aceptable. 3.6	Cumple 10 o más tareas completas, sin errores. 6	6
2. Uso de tecnologías	No se utilizan las tecnologías requeridas (Express, MongoDB, Multer, Socket.io). 0	Uso muy limitado o incorrecto de las tecnologías principales (por ejemplo, falta conexión a base de datos o no se usa Multer). 0.72	Uso aceptable de tecnologías, pero con integración incompleta o parcial (por ejemplo, carga de archivos sin validación, sesiones mal gestionadas). 1.44	Uso correcto y eficaz de todas las tecnologías requeridas, de forma coherente y funciona 2.4	2.4
3. Diseño visual y responsivo	No hay vistas ni diseño visual; no se utiliza Handlebars. 0	Vistas muy básicas, sin estructura visual clara; uso incorrecto o mínimo de Handlebars. 0.48	Vistas funcionales con Handlebars, incluyen datos dinámicos, aunque con poco diseño o sin aprovechamiento de layouts y parciales. 0.96	Diseño organizado, atractivo y funcional. Uso eficiente de Handlebars con layouts, parciales e interpolación de datos en todas las vistas necesarias. 1.6	1.6

4. Organización y entrega del proyecto Estructura de carpetas, uso de GitHub, documentación	No utiliza GitHub. El código está desorganizado y sin comentarios. No entrega el proyecto en ZIP. 0	Estructura de carpetas poco clara, sin uso adecuado de GitHub o sin documentación relevante. 0.6	Estructura aceptable del proyecto, repositorio funcional pero sin historial claro, documentación mínima o incompleta (README.md básico o vacío). 1.2	Proyecto organizado en estructura tipo MVC, uso correcto de GitHub con historial de commits, documentación clara en README.md incluyendo instalación y uso. 2	2
Totales					12